

12. Vấn đề N+1 Query trong Spring Boot (JPA/Hibernate)

12.1. Giới thiệu về N+1 Query Problem

N+1 Query Problem là một vấn đề hiệu suất phổ biến trong các ứng dụng sử dụng ORM (Object-Relational Mapping) như JPA/Hibernate. Vấn đề này xảy ra khi ORM thực hiện một truy vấn ban đầu để lấy một tập hợp các thực thể (N entities), sau đó thực hiện thêm N truy vấn riêng biệt để lấy dữ liệu liên quan (associated data) cho từng thực thể đó. Điều này dẫn đến tổng cộng N+1 truy vấn cơ sở dữ liệu, gây ra gánh nặng lớn cho database và làm giảm hiệu suất ứng dụng, đặc biệt khi số lượng thực thể N lớn.

Ví dụ minh họa:

Giả sử chúng ta có hai thực thể `Author` và `Book` với mối quan hệ One-to-Many (một tác giả có nhiều sách).

Java

```
@Entity
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "author", fetch = FetchType.LAZY)
    private List<Book> books;

    // Getters and Setters
}

@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id")
    private Author author;
}
```

```
private Author author;

// Getters and Setters
}
```

Nếu bạn muốn lấy tất cả các tác giả và sau đó truy cập danh sách sách của từng tác giả:

Java

```
// Trong một Service method
@Transactional(readOnly = true)
public List<Author> getAllAuthorsWithBooks() {
    List<Author> authors = authorRepository.findAll(); // Query 1: SELECT *
    FROM author
    for (Author author : authors) {
        // Khi truy cập author.getBooks(), Hibernate sẽ thực hiện một truy
        vấn riêng
        // cho mỗi tác giả để lấy sách của họ.
        // Query 2: SELECT * FROM book WHERE author_id = ? (cho tác giả 1)
        // Query 3: SELECT * FROM book WHERE author_id = ? (cho tác giả 2)
        // ...
        // Query N+1: SELECT * FROM book WHERE author_id = ? (cho tác giả N)
        author.getBooks().size(); // Kích hoạt tải dữ liệu (lazy loading)
    }
    return authors;
}
```

Đây chính là vấn đề N+1 Query. Một truy vấn để lấy N tác giả, và N truy vấn khác để lấy sách cho từng tác giả.

12.2. Cách phát hiện N+1 Query

Cách tốt nhất để phát hiện N+1 Query là bật logging cho các câu lệnh SQL của Hibernate.

Trong `application.yml` :

YAML

```
spring:
  jpa:
    show-sql: true
    properties:
      hibernate:
```

```
format_sql: true
use_sql_comments: true
logging:
  level:
    org.hibernate.SQL: DEBUG
    org.hibernate.type.descriptor.sql.BasicBinder: TRACE # Hiển thị giá trị tham số
```

Khi chạy ứng dụng và thực hiện các thao tác truy vấn, bạn sẽ thấy nhiều câu lệnh `SELECT` được thực thi liên tiếp cho cùng một loại dữ liệu liên quan.

12.3. Các giải pháp khắc phục N+1 Query

Có nhiều cách để giải quyết vấn đề N+1 Query, chủ yếu là bằng cách thay đổi chiến lược tải dữ liệu (fetching strategy).

12.3.1. Sử dụng `FetchType.EAGER` (Không khuyến khích cho List/Set)

Bạn có thể thay đổi `FetchType` của mối quan hệ thành `EAGER` :

Java

```
@Entity
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "author", fetch = FetchType.EAGER) // Thay đổi
    // thành EAGER
    private List<Book> books;

    // Getters and Setters
}
```

Ưu điểm: Đơn giản, không cần thay đổi code truy vấn.

Nhược điểm:

- **Performance Degradation:** Hibernate sẽ luôn tải tất cả các mối quan hệ `EAGER` ngay lập tức, ngay cả khi bạn không cần chúng. Điều này có thể dẫn đến việc tải quá nhiều dữ liệu không cần thiết, làm chậm ứng dụng và tốn bộ nhớ.
- **Cartesian Product (Đa số trường hợp):** Khi có nhiều mối quan hệ `EAGER` trong một truy vấn, Hibernate có thể tạo ra một Cartesian Product, dẫn đến việc trả về số lượng bản ghi lớn hơn nhiều so với mong đợi, gây ra `MultipleBagFetchException` nếu bạn sử dụng `List` hoặc `Set` không có thứ tự.
- **Không linh hoạt:** Không thể thay đổi chiến lược tải dữ liệu cho từng trường hợp sử dụng cụ thể.

Kết luận: `FetchType.EAGER` thường chỉ nên sử dụng cho các mối quan hệ `ManyToOne` hoặc `OneToOne` mà bạn chắc chắn luôn cần dữ liệu liên quan, và không nên sử dụng cho các mối quan hệ `OneToMany` hoặc `ManyToMany`.

12.3.2. Sử dụng `JOIN FETCH` trong JPQL/HQL

Đây là cách hiệu quả và được khuyến nghị nhất để giải quyết N+1 Query. Bạn sử dụng `JOIN FETCH` trong các truy vấn JPQL (hoặc HQL) để chỉ định rằng dữ liệu liên quan nên được tải cùng với thực thể chính trong một truy vấn duy nhất.

Java

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface AuthorRepository extends JpaRepository<Author, Long> {

    @Query("SELECT a FROM Author a JOIN FETCH a.books")
    List<Author> findAllAuthorsWithBooksJoinFetch();

    @Query("SELECT a FROM Author a JOIN FETCH a.books WHERE a.name = :name")
    List<Author> findAuthorByNameWithBooksJoinFetch(String name);
}
```

Khi bạn gọi `findAllAuthorsWithBooksJoinFetch()`, Hibernate sẽ tạo một truy vấn `SELECT` duy nhất với `LEFT JOIN` để lấy cả tác giả và sách của họ, tránh được vấn đề N+1.

Ưu điểm:

- **Hiệu quả:** Tải tất cả dữ liệu cần thiết trong một truy vấn duy nhất.
- **Linh hoạt:** Bạn có thể chọn tải dữ liệu liên quan chỉ khi cần thiết cho một trường hợp sử dụng cụ thể.
- **Tránh Cartesian Product:** Khi sử dụng `JOIN FETCH` với `List` hoặc `Set`, Hibernate sẽ xử lý để tránh Cartesian Product bằng cách sử dụng `DISTINCT` hoặc xử lý trong bộ nhớ.

Nhược điểm: Có thể phức tạp hơn một chút khi viết truy vấn.

12.3.3. Sử dụng `@EntityGraph`

`@EntityGraph` là một annotation của JPA cho phép bạn định nghĩa một biểu đồ thực thể (entity graph) để chỉ định các mối quan hệ nào nên được tải cùng với thực thể chính. Nó cung cấp một cách khai báo để thực hiện `JOIN FETCH` mà không cần viết JPQL.

Java

```
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface AuthorRepository extends JpaRepository<Author, Long> {

    @EntityGraph(attributePaths = {"books"})
    List<Author> findAllAuthorsWithBooksEntityGraph();

    @EntityGraph(attributePaths = {"books"})
    List<Author> findByName(String name);
}
```

Ưu điểm:

- **Khai báo:** Dễ sử dụng và đọc hơn so với viết JPQL `JOIN FETCH` trực tiếp.

- **Tái sử dụng:** Có thể định nghĩa nhiều `EntityGraph` cho cùng một thực thể để phục vụ các trường hợp sử dụng khác nhau.
- **Tránh Cartesian Product:** Tương tự `JOIN FETCH`, Hibernate sẽ xử lý để tránh Cartesian Product.

Nhược điểm: Ít linh hoạt hơn `JOIN FETCH` khi cần các điều kiện phức tạp trong mệnh đề `JOIN`.

12.3.4. Sử dụng `BatchSize` hoặc `FetchMode.SUBSELECT`

Đây là các giải pháp cấp độ Hibernate, không phải JPA chuẩn, nhưng rất hữu ích.

@BatchSize :

Annotation `@BatchSize` được đặt trên mối quan hệ (ví dụ: `@OneToMany`) và chỉ định số lượng các thực thể con sẽ được tải trong một truy vấn batch. Hibernate sẽ nhóm các ID của thực thể cha và tải các thực thể con trong một truy vấn `IN` duy nhất.

Java

```
import org.hibernate.annotations.BatchSize;

@Entity
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "author", fetch = FetchType.LAZY)
    @BatchSize(size = 10) // Tải 10 danh sách sách cùng một lúc
    private List<Book> books;

    // Getters and Setters
}
```

Khi bạn truy cập `author.getBooks()` lần đầu tiên, Hibernate sẽ tải sách cho 10 tác giả cùng một lúc (nếu có đủ). Điều này giảm số lượng truy vấn từ N thành $N/\text{BatchSize} + 1$.

@Fetch(FetchMode.SUBSELECT) :

Annotation `@Fetch(FetchMode.SUBSELECT)` được đặt trên mối quan hệ và yêu cầu Hibernate sử dụng một truy vấn con (subselect) để tải tất cả các thực thể con liên quan trong một truy vấn duy nhất.

Java

```
import org.hibernate.annotations.Fetch;
import org.hibernate.annotations.FetchMode;

@Entity
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "author", fetch = FetchType.LAZY)
    @Fetch(FetchMode.SUBSELECT) // Tải tất cả sách trong một subselect
    private List<Book> books;

    // Getters and Setters
}
```

Ưu điểm của `@BatchSize` và `FetchMode.SUBSELECT` :

- **Tự động:** Không cần thay đổi truy vấn JPQL/HQL.
- **Hiệu quả:** Giảm đáng kể số lượng truy vấn.

Nhược điểm:

- **Hibernate-specific:** Không phải là một phần của JPA chuẩn.
- `FetchMode.SUBSELECT` có thể không hiệu quả bằng `JOIN FETCH` cho các tập dữ liệu lớn hoặc các truy vấn phức tạp.

12.3.5. Sử dụng DTOs và Projection

Khi bạn chỉ cần một phần dữ liệu từ các thực thể liên quan, việc sử dụng DTOs (Data Transfer Objects) hoặc Projection là một cách hiệu quả để tránh tải toàn bộ thực thể và các mối quan hệ không cần thiết.

Ví dụ: AuthorWithBooksDto

Java

```
public class AuthorWithBooksDto {
    private Long id;
    private String name;
    private List<BookDto> books;

    public AuthorWithBooksDto(Long id, String name, List<BookDto> books) {
        this.id = id;
        this.name = name;
        this.books = books;
    }

    // Getters and Setters
}

public class BookDto {
    private Long id;
    private String title;

    public BookDto(Long id, String title) {
        this.id = id;
        this.title = title;
    }

    // Getters and Setters
}
```

Repository với JPQL Constructor Expression:

Java

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface AuthorRepository extends JpaRepository<Author, Long> {

    @Query("SELECT new com.example.myapp.dto.AuthorWithBooksDto(a.id, a.name, b.id, b.title) FROM Author a JOIN a.books b")
```



```
List<AuthorWithBooksDto> findAllAuthorsWithBooksDto();  
}
```

Ưu điểm:

- **Tải chính xác dữ liệu cần thiết:** Chỉ tải các cột cần thiết, giảm thiểu dữ liệu truyền tải.
- **Tránh N+1:** Tải dữ liệu liên quan trong một truy vấn duy nhất.
- **Tách biệt:** Tách biệt model nghiệp vụ khỏi API contract.

Nhược điểm: Cần tạo các lớp DTO và viết JPQL phức tạp hơn.

12.4. Bản chất của N+1 Query Problem và giải pháp

Bản chất của N+1 Query Problem nằm ở sự không khớp giữa mô hình đối tượng (object model) của ORM và mô hình quan hệ (relational model) của cơ sở dữ liệu, đặc biệt khi sử dụng chiến lược tải dữ liệu lười biếng (lazy loading) mặc định. Khi một mối quan hệ được đánh dấu là **LAZY**, ORM sẽ trì hoãn việc tải dữ liệu liên quan cho đến khi nó thực sự được truy cập. Điều này hiệu quả cho các trường hợp riêng lẻ, nhưng khi bạn lặp qua một tập hợp lớn các thực thể cha và truy cập mối quan hệ con cho từng thực thể, nó sẽ tạo ra N truy vấn riêng biệt, dẫn đến vấn đề N+1.

Các giải pháp khắc phục đều tập trung vào việc chuyển đổi chiến lược tải dữ liệu từ lazy loading sang eager loading (tải ngay lập tức) một cách có kiểm soát, hoặc bằng cách tải dữ liệu theo lô (batch fetching), hoặc bằng cách chỉ tải những gì thực sự cần thông qua Projection/DTOs. Mục tiêu là giảm số lượng truy vấn đến cơ sở dữ liệu, tối ưu hóa việc sử dụng tài nguyên và cải thiện hiệu suất tổng thể của ứng dụng. Việc lựa chọn giải pháp phù hợp phụ thuộc vào ngữ cảnh sử dụng, số lượng dữ liệu, và mức độ phức tạp của mối quan hệ.